

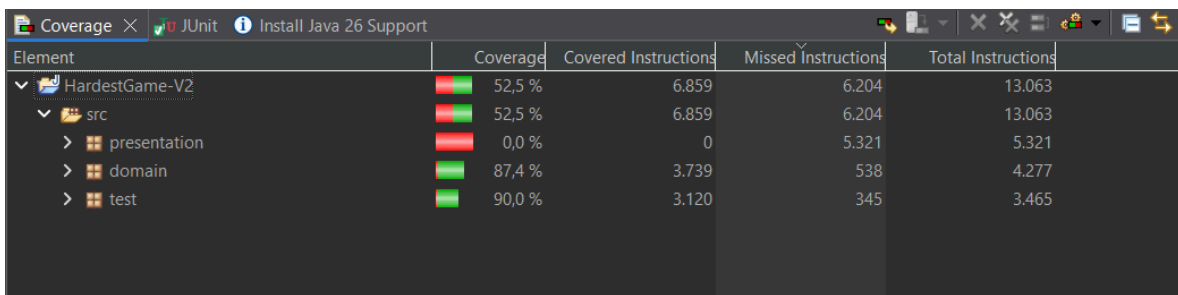
Informe inicial de Análisis Dinámico: Pruebas y Cobertura de Código

YAZID ALEJANDRO SÁNCHEZ SÁNCHEZ

JOSE ALEJANDRO MARTÍNEZ ARIAS

Análisis Cobertura

El análisis dinámico realizado mediante la herramienta de Test Coverage arroja una cobertura total de instrucciones del 52.5% global. Debemos tener en cuenta que el paquete presentation (con 0.0% de cobertura) influye en este resultado general. Al estar relacionado enteramente con la interfaz gráfica de usuario (Componentes de Java Swing), no se realizaron pruebas sobre este paquete ya que no es el alcance buscado con las pruebas unitarias; nuestro objetivo central es probar la buena lógica del negocio y del dominio.



Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
HardestGame-V2	52,5 %	6.859	6.204	13.063
src	52,5 %	6.859	6.204	13.063
presentation	0,0 %	0	5.321	5.321
domain	87,4 %	3.739	538	4.277
test	90,0 %	3.120	345	3.465

Clases fundamentales del sistema muestran un buen cubrimiento, lo que garantiza que las funcionalidades principales del juego estén protegidas ante futuros cambios:

- DOPOGame.java (90.9%) y GameMode.java (92.8%): La fachada principal del juego y los modos de juego (SinglePlayer / PvP)
- LevelLoader.java (85.5%): La lógica de creación y carga de niveles a partir de archivos de texto fue validada incluso simulando excepciones de lectura. Demuestran que la lógica de creación y manejo de errores críticos está validada.
- PhysicsEngine.java (92.3%) y Tipos de Movimiento (100%): Se verifican rigurosamente físicas de rebote, límites de movimiento en diagonal e interacciones de hitboxes.
- Clases como LevelLoader.java (85.5%), GameMode.java (92.0%) y DopoGameException.java (100%)
- Lógica Principal: La clase Tower.java alcanza un 87.,4%, lo que garantiza que las funcionalidades principales del juego están protegidas ante cambios.

Esto garantiza que la jugabilidad principal del juego se ejecuta de buena manera con base a los requerimientos solicitados

Calidad de las Pruebas (Paquete Test)

Se segmentaron los test en diferentes archivos .java de pruebas, por comportamiento y responsabilidad:

1. **GameplayLogicTest:** Valida las condiciones de victoria, las colisiones en el modo PvP y la manipulación de los diferentes estados del juego.
2. **BoardElementTest:** Verifica el comportamiento unitario de los objetos del tablero, validando en consecuencia la lógica de recolección de monedas, detonación de bombas y curaciones.
3. **PlayerSkinTest:** Garantiza que los incrementos de velocidad o los escudos extras proporcionados por las diferentes apariencias (Skins) funcionen estadísticamente como se espera.
4. **PersistenceLogTest:** Contiene pruebas de integración robustas que garantizan la perfecta carga y guardado (serialización profunda) del estado del juego sin perder monedas o posiciones.

Para una interpretación correcta de este informe, se deben considerar:

- **Collectable.java (0.0%):** Es una interfaz declarativa. Hay metodos por defecto que son sobrescritos por las clases que las implementan (como las monedas), por lo que se asume erróneamente que las líneas base no se ejecutan. No representa un fallo lógico en el sistema.
- **Paquete Presentation (0.0%):** Clases como GamePanel y V2GUI requieren la instanciación y renderizado de la ventana de diálogo y gráficos en un hilo visual (Event Dispatch Thread), lo cual se excluye intencionalmente de las pruebas unitarias. De antemano, para esto tenemos las pruebas de aceptación que verifican jugabilidad del juego por parte de los usuarios y buena representación del mismo.

Análisis Detallado del Paquete Domain

Este paquete representa la lógica de dominio.

Para una interpretación correcta de este informe, se deben considerar los siguientes puntos:

- Paquete presentation: Representa la interfaz grafica con la cual el usuario actuara, por lo tanto no se hizo mucho énfasis en su verificación ,ya que implicaba representación en pantalla. Estos se podrían evidenciar y rectificar con la ejecución del juego.
- Paquete Test: A pesar de que tienen un porcentaje alto, hay que tener en cuenta que estos alteran el porcentaje de Test Coverage, ya que estos son los que validan la buena lógica del dominio, por lo tanto se podría omitir este paquete.
- Por otro lado, se rectificó que ya estamos cumpliendo con la meta esperada de un 75% en la pruebas, en las cuales cabe resaltar que las pruebas no buscaban cumplir simplemente con la métrica impuesta, lo que se busco fue realizar test de calidad sobre cumplimiento por métrica, esto por ende traía consigo un buen resultado en el porcentaje, donde cabe aclarar que es en el paquete de Domain donde se busca cumplir con el porcentaje esperado en esta clase de pruebas

Conclusiones

Se cumplió con el porcentaje esperado, lo cual indica que se tuvieron se cumplieron con los comportamientos esperados requeridos

Se logró superar la meta acordada del 75%, obteniendo un 93.1% en el paquete domain, el cual encapsula lo que se busca probar del proyecto, teniendo en cuenta la buena

Decisiones tomadas: Como se cumplió con el cubrimiento del 75% concluimos que la lógica del proyecto se cubrió de una buena manera, por lo tanto, no se realizaron nuevas pruebas, se considera que las pruebas planteadas, ya verifican la buena lógica del proyecto, siendo dinámicas y preservando en cada una de ella verificar la buena jugabilidad del juego sin buscar simplemente cumplir con las métricas esperadas.

Por otro lado, se tiene en cuenta que se implementaron pruebas de aceptación, las cuales fueron anexadas en otro informe, donde rectificamos que la buena representación del juego y casos dinámicos del juego en cuestión de comportamientos de sus elementos.